

Chapter 6. Effect Heterogeneity

This chapter introduces what is perhaps the most interesting development of causal inference applied to the industry: effect heterogeneity. Up until this point, you understood the general impact of a treatment. Now, you'll focus on finding how it can affect people differently. The idea that the treatment effect is not constant is simple, yet incredibly powerful. Knowing which units respond better to a certain treatment is key in deciding who gets it. Effect heterogeneity offers a causal inference approach to the cherished idea of personalization.

You'll start by understanding effect heterogeneity on a theoretical level, what the challenges are in estimating effect heterogeneity, and how you can expand what you already learned to work around those challenges. Next, you'll see that estimating heterogeneous effects is closely related to predictive problems, which are already very familiar to data scientists. Consequently, you'll see how the idea of cross-validation and model selection still applies in treatment heterogeneity models. However, validating your effect estimate is much more challenging than evaluating a simple predictive model, which is why you'll see some novel ideas on how to do it.

The chapter closes with some guidelines and examples on how to use effect heterogeneity to guide decision making. Although

not exhaustive, I hope those examples will inform you on how to use these ideas on your own business problems.

From ATE to CATE

So far, every time you've estimated the causal impact of a treatment, it was mostly the average treatment effect:

$$\tau = E[Y_1 - Y_0]$$

or the continuous treatment equivalent:

$$\tau = E[y'(t)]$$

where $y'(t)$ is the derivative of the treatment response function. That is, you've learned techniques to uncover the general effectiveness of a treatment. ATE estimation is the bedrock of causal inference. It's a super useful tool for the decision-making problem that is referred to as program evaluation: when you want to know if you should roll out a treatment to the entire population or not.

Now, it's time to learn how to inform another type of decision: *which unit* should you treat? For that, you'll need to allow the decision to change from one unit to another. For example, it might be beneficial to give a discount coupon to one customer but not to another, since one customer might be more sensitive to discounts. Or, it might make sense to prioritize a vaccine to one group over another, as those would benefit

more from such treatment. In this type of situation, personalization is key.

One way to achieve this personalization is by taking effect heterogeneity into account, which involves estimating the conditional average treatment effect (CATE). By considering the unique characteristics of each unit, you can determine the most effective treatment for that particular case:

$$E[Y_1 - Y_0|X] \text{ or } E[y'(t)|X]$$

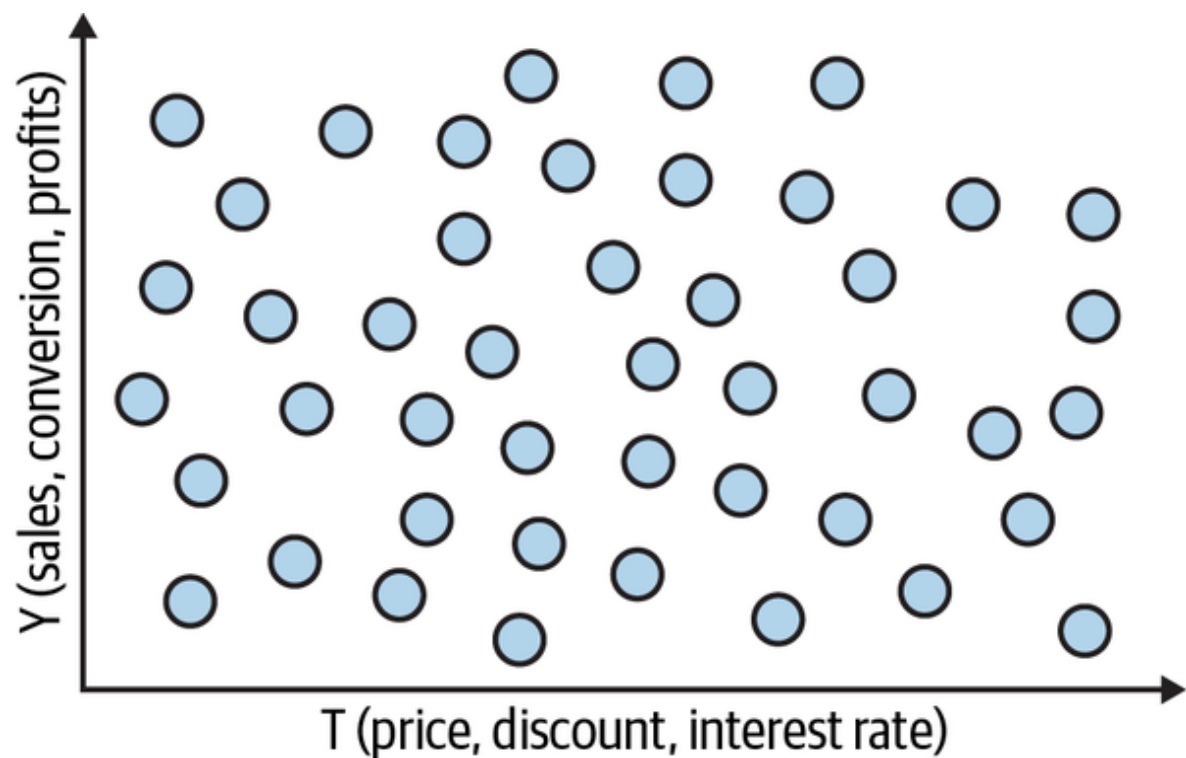
The conditioning on X means that you now allow the treatment effect to be different depending on the characteristics, as defined by the covariates X of each unit. Again, here, you believe that not all entities respond equally well to the treatment and you want to leverage that heterogeneity. You want to treat only the right units (in the binary case) or figure out the optimal treatment dosage for each unit (in the continuous case).

For instance, if you are a bank that has to decide the loan each customer is eligible for, you can be sure that it's not a good idea to give loads of money to everyone, although it might be reasonable for some. You will have to be smart with your treatment (loan amount). Perhaps, depending on the customer credit score, you can figure out what the proper loan amount is. Of course, you don't need to be a big institution to leverage personalization. There's no shortage of examples where it applies. What days of the year should you do sales? How much

should you charge for a product? How much exercise is too much exercise for each person?

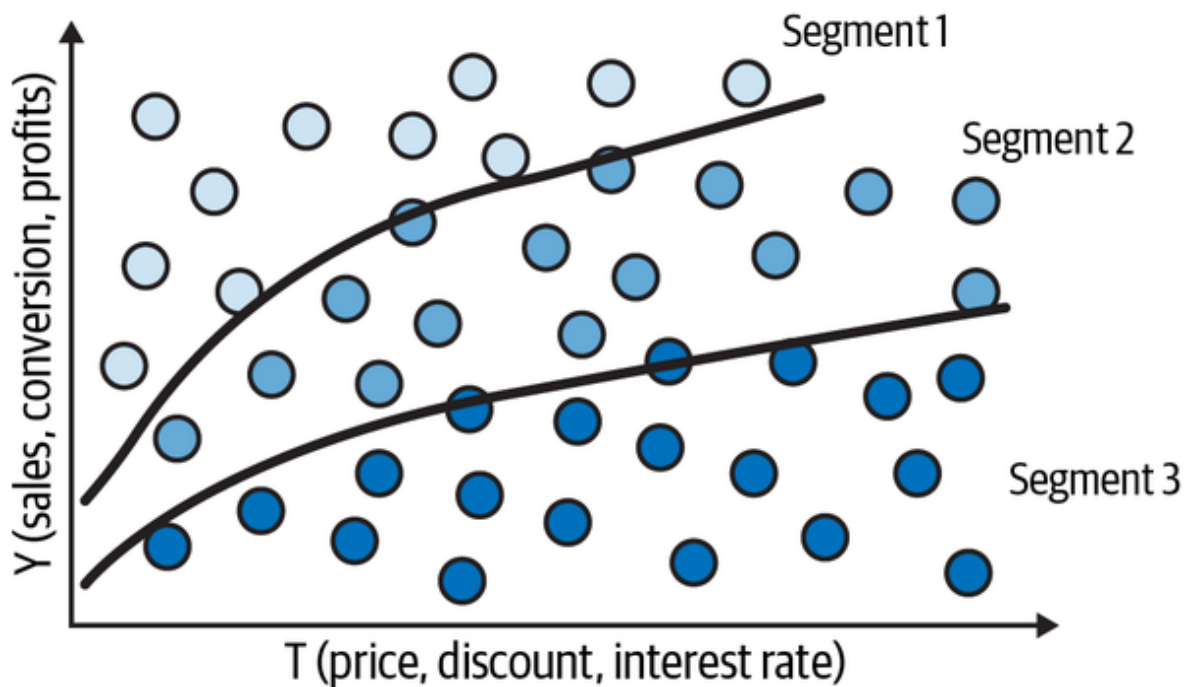
Why Prediction Is Not the Answer

Think of it this way. You have a bunch of customers and a treatment (price, discount, loan...) and you want to personalize the treatment—for example, give different discounts to different customers. And say you can organize your customers in the following treatment by outcome plot:



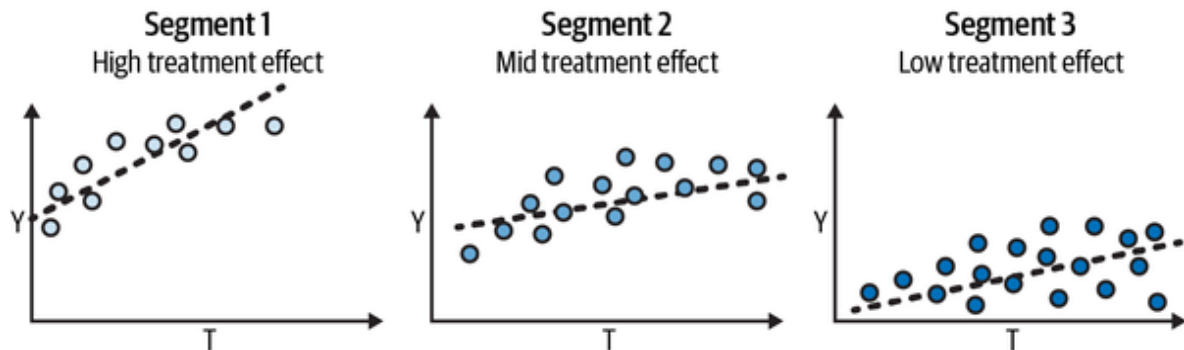
You can think about the personalization task as problem of customer segmentation. You want to create groups of customers based on how they respond to your treatment. For

example, say you want to find customers who respond well to discounts and customers who respond poorly to them. Well, the customer's response to a treatment is given by the conditional treatment effect $\frac{\delta Y}{\delta T}$. So, if you could somehow estimate that for each customer, you would be able to group those who respond great to the treatment (high treatment effect) and those who don't respond very well to it. If you did that, you would split the customer space somewhat like the following image:

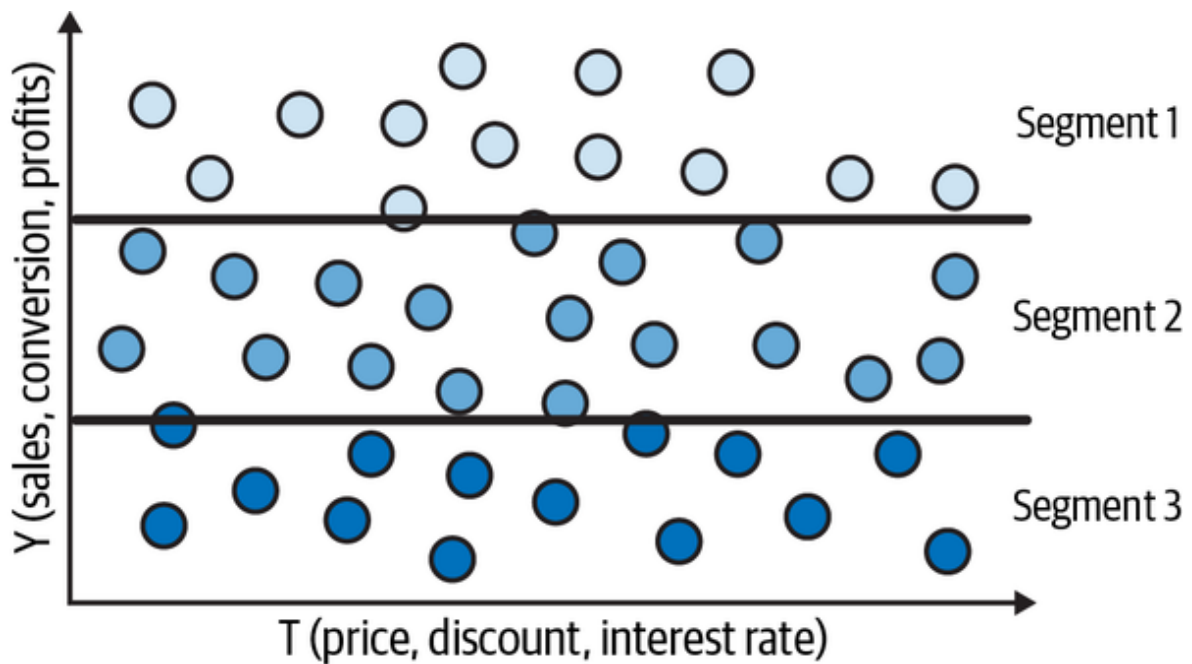


That would be wonderful because now you would be able to estimate different treatment effects on each group. Again, since the effect is just the slope of the treatment response function, if you can produce groups where that slope differs,

entities on those partitions will have different responsiveness to the treatment:



Now, contrast this with what you would get with a traditional machine learning approach. You would probably try to predict Y , rather than the derivative $\frac{\delta Y}{\delta T}$ for each unit. This would essentially partition the space on the y-axis, assuming that your predictive model can approximate the target well. However, this wouldn't necessarily lead to groups with different treatment effects. Which is why simply predicting the outcome is not always useful for decision making:



OK, you might say, I get that I have to estimate the effect, instead of just predict the outcome, but it's kind of tricky. How can I predict the slope $\frac{\delta Sales}{\delta Discount}$ if I can't see it?

That's a good point. Unlike the raw outcome Y , slopes (or rate of changes) are essentially nonobservable on a unit level. For you to see the individual slopes, you would have to observe each unit under different treatment levels and calculate how the outcome changes for each of those treatments:

$$\frac{\delta Y_i}{\delta T_i} \approx \frac{Y(T_i) - Y(T_i + \epsilon)}{T_i - (T_i + \epsilon)}$$

This is the fundamental problem of causal inference all over again. You can't ever see the same unit under different treatment conditions. So, what can you do?

CATE AND ITE

Keep in mind that the CATE is different from the individual treatment effect (ITE). For instance, suppose you have two groups, $x = 1$ and $x = 2$, each with 4 units, and you want to know the effect of a new drug on an illness that usually kills 50% of those with it. For group $x = 1$, the medicine is detrimental to one patient, killing it, but it saves another. For the group $x = 2$, the effect of the drug is null, and 1 of them dies (remember that the illness kills 50%). In both groups, the CATE is 0.5, but the ITE of no single unit is 0.5.

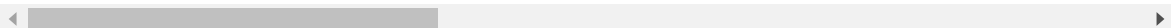
CATE with Regression

I think you probably saw it coming: as with most things in applied causal inference, the answer tends to start with linear regression. But before going that route, let's make things a bit more tangible. Pretend that you work for a chain of restaurants that operate across the country. A key component of this business is understanding *when* it should give discounts to its customers. For this reason, the company ran a nationwide experiment that lasted three years, where it randomized discounts in six different restaurants in the chain. The data is stored in the following data frame:

```
In [1]: data = pd.read_csv("../data/daily_restau:  
  
data.head()
```



	rest_id	day	month
0	0	2016-01-01	1
1	0	2016-01-02	1
2	0	2016-01-03	1
3	0	2016-01-04	1
4	0	2016-01-05	1



Your goal here is to understand when it is the best time to give discounts. In this data, you have one row per restaurant and day combination. This is a bit different from most of the examples used in this book, where the unit was customers. Now, the unit is a day–restaurant combination. Even so, you can still apply the same reasoning from before, only instead of treating customers, you’ll “treat” (give a discount) days instead of customers. You could also have a different price at each restaurant at each day, but let’s simplify the problem to keeping prices consistent across restaurants.

You can frame this business problem as a CATE estimation problem. If you can create a model that outputs the sales

sensitivity to discount for each day and covariate, that is:

$$\frac{\partial}{\partial t} E[\text{Sales}(t)|X],$$

then, you can use that model to decide when to give a discount and how much discount to give.

CATE IDENTIFICATION

Throughout this chapter, you won't have to worry too much about identification, since the treatment is randomized in the evaluation set. However, the whole idea of estimating the CATE is based on making $E[\text{Sales}(t)|X] = E[\text{Sales}|T = t, X]$.

Now that you have something more tangible to work with, let's see how regression can help you. Recall that you were left in a complicated situation. You need to predict $\frac{\delta Y_i}{\delta T_i}$, which is sadly not observable. So it's not like you could simply use an ML algorithm and plug that as its target. But maybe you don't need to observe $\frac{\delta Y_i}{\delta T_i}$ in order to predict it.

For instance, let's say you fit the following linear model to your data:

$$y_i = \beta_0 + \beta_1 t_i + \beta_2 X_i + e_i$$

If you differentiate it on the treatment, you will end up with the following:

$$\frac{\delta y_i}{\delta t_i} = \beta_1$$

which is the ATE, in the case of a randomized treatment.

Since you can estimate the preceding model to get $\widehat{\beta}_1$, you might even go ahead to say that *you can predict slopes even though you can't observe them*. In the example, it is a rather simple prediction. You are predicting the constant value $\widehat{\beta}_1$ for everyone. That's something, but not quite what you want. That's the ATE, not the CATE. This doesn't help you in your task of figuring out when to give discounts, simply because every unit (day and restaurant combination) gets the same slope prediction.

To improve upon it, you can do the following simple change:

$$y_i = \beta_0 + \beta_1 t_i + \beta_2 X_i + \beta_3 t_i X_i + e_i$$

which would, in turn, give you the following slope prediction:

$$\frac{\widehat{\delta y_i}}{\delta t_i} = \widehat{\beta}_1 + \widehat{\beta}_3 X_i$$

where β_3 is a vector coefficient for the features in X . Now you are getting somewhere! Each entity defined by a different X_i will have a different slope prediction. In other words, the slope prediction will change as X changes. Intuitively speaking, including the interaction between the treatment and the covariates allows the model to learn how the effect changes by those same covariates. This is how regression can give you a way of estimating the CATE, even though you can't predict it directly.

Enough of theory for now. Let's see how to code this up. First, you need to define the covariates. In this example, the covariates are basically date-specific features, like the month, the day of the week, and if it is a holiday. I'm also including the average competitor's price, as this will probably affect how customers respond to discounts in each restaurant.

Once you have the covariates, you need to interact them with the treatment. The `*` operator does exactly that. It creates an additive term for the left and right side plus an interaction term. For example, `a*b` will include the terms `a`, `b` and `a * b` in your regression. In your example, this would result in the following:

$$sales_i = \beta_0 + \beta_1 discount_i + \beta_2 X_i * discount_i + \beta_3 X_i + e_i$$

```
In [2]: import statsmodels.formula.api as smf

X = ["C(month)", "C(weekday)", "is_holiday"]
regr_cate = smf.ols(f"sales ~ discounts + {' + '.join(X)}",
                    data=data).fit()
```

* AND : OPERATORS

If you only want the multiplicative term, you can use the `:` operator inside the formula.

Once you've estimated the model, the predicted slope can be extracted from the parameter estimates:

$$\frac{\widehat{\delta sales_i}}{\widehat{\delta discounts_i}} = \widehat{\beta_1} + \widehat{\beta_3} X_i$$

where β_1 is the discount coefficient and β_3 is the vector for the interaction coefficients. You could just extract those parameters from the fitted model, but an easier way to get the slope predictions is to use the definition of the derivative:

$$\frac{\delta y}{\delta t} = \frac{y(t + \epsilon) - y(t)}{(t + \epsilon) - t}$$

with ϵ going to zero. You can approximate this definition by replacing ϵ with 1:

$$\frac{\delta y}{\delta t} \approx \hat{y}(t + 1) - \hat{y}(t)$$

where $\hat{y}$ is given by your model's predictions. Since this is a linear model, the approximation is exact.

In other words, you'll make two predictions with your models: one passing the original data and another passing the original data but with the treatment incremented by one unit. The difference between those predictions is your CATE prediction. Here is what this looks like with some code:

```
In [3]: ols_cate_pred = (  
        regr_cate.predict(data.assign(discount=1))  
        - regr_cate.predict(data)
```

)

OK, you have your CATE model and its predictions. But there is still a lurking question: how good is it? In other words, how can you evaluate this model? As you can probably tell, comparing actuals and predicted values won't do here, since the actual treatment effect is not observed at a unit level.



PRICE DISCRIMINATION

In the microeconomic literature, the example used in this chapter is what's called *price discrimination*. Despite the bad-sounding name, it simply means that firms can discriminate consumers into those who are willing to pay more and charge them more. A very well known example of price discrimination is when an airline company changes the airfare depending on how far in advance the ticket is bought: customers who need to book a flight for next week can expect to pay much more than those booking one for next year. This is called *intertemporal price discrimination*, since the company manages to distinguish the price sensitivity of customers based on time. It is a very similar situation to the restaurant example you saw in this chapter.

A more infamous example would be when a wine company sells the same exact wine in two different bottles, one marketed as premium at a much steeper price and one marketed as average, sold at a more modest value. Yet a third way of price discriminating is when you have half-price entry tickets for students. In this case, the company knows that students make less money on average, meaning they have less to spend.

Evaluating CATE Predictions

If you come from a traditional data science background, you can probably see that this sort of CATE prediction looks a lot like regular machine learning prediction, but with a sneaky target that is not observed at a unit level. This means that a lot of the model evaluation techniques used in traditional machine learning—like cross-validation—still apply here, while others will need some adaptation.

So, to keep with tradition, let's split the data into a train and a test set. Since you have a time dimension, let's use that. The train will contain data from 2016 and 2017 and the test, from 2018 onward:

```
In [4]: train = data.query("day<' 2018-01-01' ")
        test = data.query("day>=' 2018-01-01' ")
```

Now, let's refit the regression model for CATE from before, but using only the training data for estimation and making predictions on the test set:

```
In [5]: X = ["C(month)", "C(weekday)", "is_holiday"]
        regr_model = smf.ols(f"sales ~ discount + {X}",
                             data=train).fit()

        cate_pred = (
            regr_model.predict(test.assign(discount=0))
            - regr_model.predict(test)
```



```
)
```

To make things interesting, let's benchmark this regression model with a purely predictive machine learning model. This ML model simply tries to predict the outcome Y :

```
In [6]: from sklearn.ensemble import GradientBoostingRegressor

X = ["month", "weekday", "is_holiday", "walked_area"]
y = "sales"

np.random.seed(1)
ml_model = GradientBoostingRegressor(n_estimators=100)

ml_pred = ml_model.predict(test[X])
```

Finally, let's also include a very bad model in our comparisons. This model simply outputs random numbers between -1 and 1 . It is obviously nonsense, but an interesting benchmark to keep an eye on. Ultimately, you want to know if allocating the treatment by a CATE model will be better than simply at random, which is what this last model does.

For convenience, I'm storing everything in a new data frame,

```
test_pred :
```

```
In [7]: np.random.seed(123)

        test_pred = test.assign(
            ml_pred=ml_pred,
            cate_pred=cate_pred,
            rand_m_pred=np.random.uniform(-1, 1,
            )
```

	rest_id	day	sales
731	0	2018-01-01	251.5
732	0	2018-01-02	541.0
733	0	2018-01-03	431.0
734	0	2018-01-04	760.0
735	0	2018-01-05	78.0

Once you have your models, it's time to figure out how to evaluate and compare them. Namely, you'll have to deal with the fact that the ground truth is non-observable. As you'll soon see, the trick is to realize that even though you can't measure

the treatment effect for a single individual, you can estimate it for very small groups. Hence, if you wish to come up with a way to evaluate your model in terms of CATE, you'll have to rely on group-level metrics.

Effect by Model Quantile

The idea of making CATE models came from the necessity of finding which units are more sensitive to the treatment with the goal of allocating the treatment more efficiently. It came from a desire for personalization. If that is the goal, it would be very useful if you could somehow order units from more sensitive to less sensitive. And since you have the predicted CATE, you can order units by that prediction and hope it also orders them by the real CATE. Sadly, you can't evaluate that ordering on a unit level. But, what if you don't need to? What if, instead, you evaluate groups defined by the ordering?

First, recall that if the treatment is randomly assigned, you don't have to worry about confounding bias here. Estimating the effect for a group of units is easy. All you need is to compare the outcome between the treated and untreated. Or, more generally, run a simple regression of Y on T inside that group:

$$y_i = \beta_0 + \beta_1 t_i + e_i | X = x$$

From the theory on simple linear regression, you know that:

$$\widehat{\beta}_1 = \frac{\sum (t_i - \bar{t}) y_i}{\sum (t_i - \bar{t})^2}$$

where $\bar{t}$ is the group sample average for the treatment and $\bar{y}$ is the group sample average for the outcome.

CURRY

The curry decorator is a way to create functions that can be partially applied:

```
@curry
def addN(x, N):
    return x+N

ad5 = addN(N=5)
ad13 = addN(N=13)

print(ad5(5))
>>> 10

print(ad13(5))
>>> 18
```

To code the slope parameter estimate of a single variable regression, you can use `curry`. It's very useful when you need to create functions that accept a data frame as its only argument:

```
In [8]: from toolz import curry

        @curry
        def effect(data, y, t):
            return (np.sum((data[t] - data[1-t]) * y) /
                    np.sum((data[t] - data[1-t])))
```

Applying this function to the entire test set yields the ATE:

```
In [9]: effect(test, "sales", "discounts")
```

```
Out[9]: 32.16196368039615
```

But that is not what you want. Instead, you want to know if the models you've just fitted can create partitions in the data that separate units into those that are more sensitive to the treatment from those that are less sensitive. For that, you can segment the data by quantiles of your model's prediction and estimate the effect for each quantile. If the estimated effect in each quantile is ordered, you know that the model is also good at ordering the true CATE.

RESPONSE CURVE SHAPE

Here the effect is defined as estimated slope of regressing Y on T . If you think this is not a good effect metric, you could use others. For instance, if you think the response function is concave, you could define the effect as the slope of regressing Y on $\log(T)$ or $\sqrt{T}$. If you have a binary outcome, it might make sense to use the parameter estimate of a logistic regression, instead of the one from a linear regression. The key thing here is to see that, if T is continuous, you have to summarize the entire treatment response function into a single effect number.

Let's code a function to calculate the effect by quantile. It first uses `pd.qcut` to segment the data by `q` quantiles (10, by default). I'm wrapping it in `pd.IntervalIndex` to extract the midpoint of each group returned by `pd.qcut`. The rounding is just so the results look prettier.

Then, I'm creating a column in the data with these groups, partitioning the data by them and estimating the effect in each partition. For this last step, I'm using the `.apply(...)` method from pandas. This method takes a function that has a data frame as an input and outputs a number:

`f(DataFrame) -> float`. Here is where the `effect` function you created earlier comes into play. You can call it passing just the outcome and treatment

arguments. This will return a partially applied `effect` function that has the data frame as the only missing argument. It is the type of function `.apply(...)` expects.

The result of using this function in the `test_pred` data frame is a column where the indexes are the quantiles of your model's prediction and the values, the treatment effect in that quantile:

```
In [10]: def effect_by_quantile(df, pred, y, t,

    # makes quantile partitions
    groups = np.round(pd.IntervalIndex

    return (df
        .assign(**{f"{pred}_quantile":
        .groupby(f"{pred}_quantile"
        # estimate the effect on ea
        .apply(effect(y=y, t=t)))

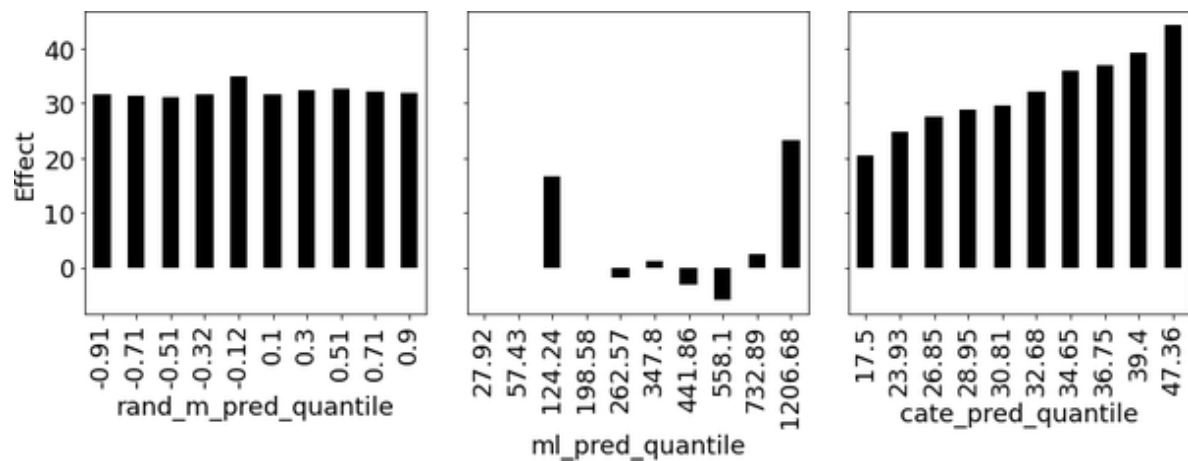
    effect_by_quantile(test_pred, "cate_pre
```

```
Out[10]: cate_pred_quantile
          17.50      20.494153
          23.93      24.782101
          26.85      27.494156
```

```
28.95      28.833993
30.81      29.604257
32.68      32.216500
34.65      35.889459
36.75      36.846889
39.40      39.125449
47.36      44.272549
dtype: float64
```

Notice how the estimated effect in the first quantile is lower than the estimated effect in the second quantile, which is lower than the estimated effect in the third quantile, and so on. This is evidence that your CATE prediction is indeed ordering the effect: days with lower predictive value also have low sensitivity to discount and vice versa. Also, the midpoint prediction in each quantile (the index in the preceding column) is very close to the estimated effect for the same quantile. This means that your CATE model not only orders the true CATE quite well, but it also manages to predict it correctly. In other words, you have a calibrated model for the CATE.

Next, so you have other models to compare against, you can apply the same function, but passing the predictive ML model and the random model. The following plot shows the effect by quantile for the three models defined earlier:



First, look at the random model (`rand_m_pred`). It has roughly the same estimated effect in each of its partitions. You can already see just by looking at the plot that it won't help you with personalization since it can't distinguish between days with high and low discount sensitivity. The effect in all of its partitions is just the ATE. Next, consider the ML predictive model, `ml_pred`. That model is a bit more interesting. It looks like groups with high sales predictions and low sales predictions are both more sensitive to discounts. It doesn't exactly produce an ordering score, though, but you could use it for personalization, maybe giving more discounts when sales predictions are either very high or very low, as those indicate high treatment sensitivity.

Finally, look at the CATE model you got from regression, `cate_pred`. The group with low CATE prediction has indeed lower CATE than the groups with high CATE predictions. It looks like this model can distinguish high from low effects pretty well. You can tell by the staircase shape of its

effect by a quantile plot. In general, the steeper the staircase shape, the better the model in terms of ordering CATE.

In this example, it is pretty clear which model is better in terms of ordering sensitivity to discount. But if you have two decent models, the comparison might not be that clear cut. Also, visual validations are nice, but not ideal if you want to do model selection (like hyperparameter tuning or feature selection). Ideally, you should be able to summarize the quality of your model in a single number. We'll get there, but to do so, you first need to learn about the cumulative effect curve.

Cumulative Effect

If you understood the effect by quantile plot, this next one will be pretty easy. Once again, the idea is to use your model to define groups and estimate effects inside those groups. However, instead of estimating the effect by group, you will accumulate one group on top of the other.

First, you need to sort your data by a score—usually a CATE model, but it can be anything really. Then, you'll estimate the effect on the top 1%, according to that ordering. Next, you'll add the following 1% and calculate the effect on the top 2%, and then on the top 3% and so on and so forth. The result will be a curve of effect by cumulative sample. Here is a simple code to do that:

```
In [11]: def cumulative_effect_curve(dataset, p:
                                ascending=True)

    size = len(dataset)
    ordered_df = (dataset
                  .sort_values(predict:
                  .reset_index(drop=True)

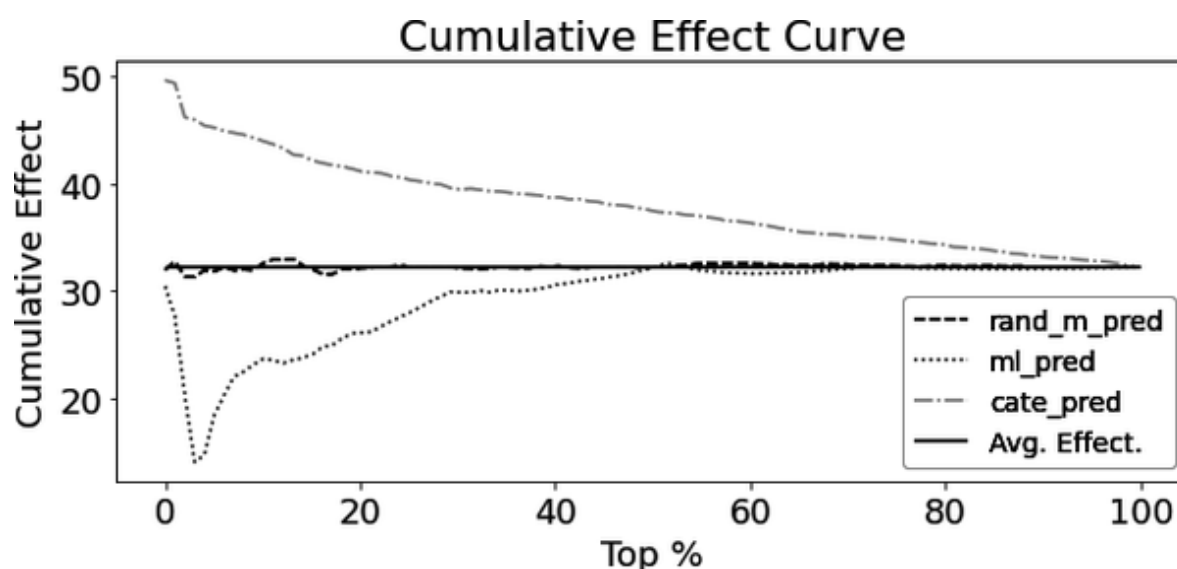
    steps = np.linspace(size/steps, size)

    return np.array([effect(ordered_df
                            for row in steps])

cumulative_effect_curve(test_pred, "cat
```

```
Out[11]: array([49.65116279, 49.37712454, 46.20
               32.46981935, 32.33428884, 32.16196368])
```

If the score you used to sort the data is also good for ordering the true CATE, the resulting curve will start very high and gradually decrease to the ATE. In contrast, a bad model will either quickly converge to the ATE or simply fluctuate around it all the time. To better understand this, here is the cumulative effect curve for the three models you've created:



First, notice how the regression CATE model starts very high and gradually converges to the ATE. For instance, if you sort your data by this model, the ATE in the top 20% will be around 42, the ATE of the top 50% would be something like 37, and the ATE of the top 100% will simply be the global effect of the treatment (ATE). In contrast, a model that simply outputs random numbers will just gravitate around the ATE and a model that reverse orders the effect will start below the ATE.

ORDER ASYMMETRY

It's important to mention that this ordering is not symmetric. That is, taking a score and reversing it won't simply flip the curve around the ATE line.

The cumulative effect curve is somewhat nicer than the effect by the quantile curve because it allows a summarization into a single number. For instance, you could compute the area between the curve and the ATE and use that to compare different models. The bigger the area, the better the model. But there is still a downside. If you do this, the beginning of

the curve will have the biggest area. But that is exactly where the uncertainty is the largest, due to the smaller sample size. Fortunately, there is a very easy fix: the cumulative gain curve.

Cumulative Gain

If you take the exact same logic from the cumulative effect curve, but multiply each point by the cumulative sample, N_{cum}/N , you get the cumulative gain curve. Now, even though the beginning of the curve will have the highest effect (for a good model, that is), it will get downscaled by the small relative size.

Taking a look at the code, what changes is that I'm now multiplying the effect by `(row/size)` at each iteration. Additionally, I can choose to normalize this curve by the ATE, which is why I'm also subtracting a normalizer from the effect at each iteration:

```
In [12]: def cumulative_gain_curve(df, prediction,
                                     ascending=False):

    effect_fn = effect(t=t, y=y)
    normalizer = effect_fn(df) if normalize else 0

    size = len(df)
    ordered_df = (df
                  .sort_values(prediction)
                  .iloc[0:size])
```

```

        .reset_index(drop=True)

    steps = np.linspace(size/steps, size, steps)
    effects = [(effect_fn(ordered_df.quantile(
        row/size, axis=0) - normalizer) * (row/size))
               for row in steps]

    return np.array([0] + effects)

cumulative_gain_curve(test_pred, "cate")

```

```

Out[12]: array([ 0.          ,  0.50387597,  0.98420403,
                32.00615008, 32.16196368])

```

SEE ALSO

If you don't want to bother implementing all these functions, I've been working along with some colleagues on a Python library to handle that for you. You can simply import all the curves and their AUC from [fklearn](#) [causal module](#):

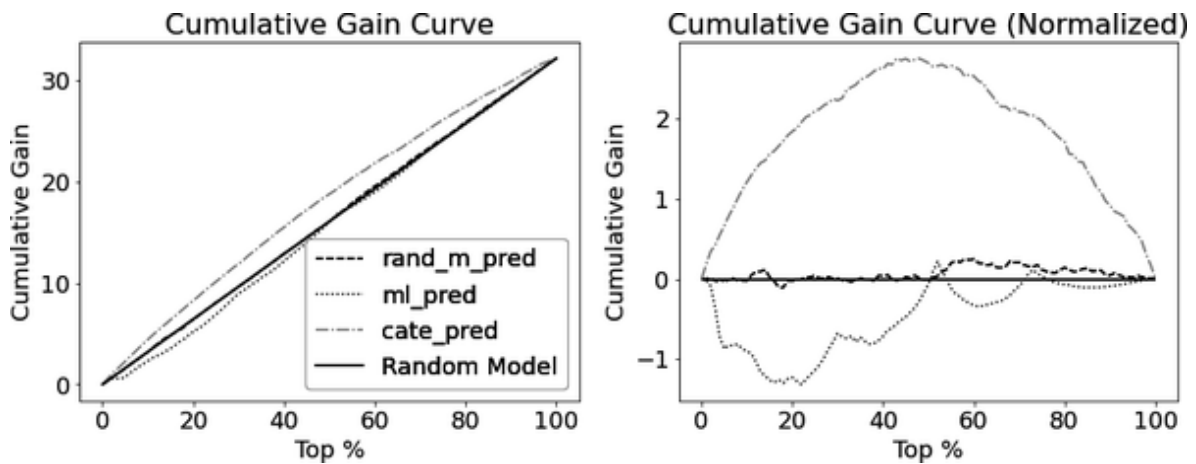
```

from fklearn.causal.validation.auc import *
from fklearn.causal.validation.curves import *

```

Both the cumulative gain and normalized cumulative gain for the three models are shown in the following image. Here, the better model in terms of ordering CATE is the one that has the

biggest area between the curve and the dashed line representing the ATE:



To summarize the model performance into a single number, you can just sum the values from the normalized cumulative gain curve. The model with the biggest value will be the best one in terms of ordering CATE. Here is the area under the curve (AUC) for the three models you've been evaluating so far. Notice that the area for the ML model is negative because it reverse-orders CATE:

```
AUC for rand_m_pred: 6.0745233598544495
AUC for ml_pred: -45.44063124684
AUC for cate_pred: 181.74573239200615
```

Again, the fact that you can condense your model's performance into a single number is amazing, as it allows for automated model selection. Still, as much as I like this last

curve, there are some caveats you need to be aware of when using them.

First, in all the curves that you saw, it's important to keep in mind that each point in this curve is an estimate, not a ground truth value. It is the estimate of the regression slope on a particular—and sometimes very small—group. And since it is a regression estimate, it depends on the relationship between T and Y being correctly specified. Even with randomization, if the relationship between the treatment and the outcome is, let's say, a log function, estimating the effect as if it were a line will yield wrong results. If you know the shape of the treatment response function, you can adjust your effect function to be the slope of $y \sim \log(t)$ instead of $y \sim t$. But you need to know the correct shape in order to do that.

Second, these curves don't really care if you get the CATE right. All they care about is if the *ordering is correct*. For example, if you take any of your models and subtract $-1,000$ from their predictions, their cumulative gain curve will remain unchanged. Hence, even if you have a biased estimator for the CATE, this bias won't show up in these curves. Now, this might not be a problem, if all you care about is prioritizing the treatment. In this case, ordering is enough. But if you do care about precisely estimating the CATE, these curves might be misleading. If you come from a data science background, you can draw a parallel between the cumulative

gain curve and the ROC curve. Similarly, a model with good ROC-AUC won't necessarily be calibrated.

Third, and perhaps most importantly, *all of the preceding methods require unconfounded data*. If you have any bias, the effects you'll estimate—in the subgroups or the ATE—will be wrong. If the treatment is not randomized, in theory, you can still use these evaluation techniques, provided you debiased the data before, by using something like orthogonalization of IPW. However, I'm a bit skeptical of this. Instead, I strongly recommend you invest in some experimental data, even if it is just a little, only for the purpose of evaluation. That way you can focus on effect heterogeneity without having to worry about confounding creeping in.

SEE ALSO

All the curves presented here are an attempt to generalize the curves traditionally used for uplift modeling, when the treatment is binary. If you want to review that literature, I recommend the papers “Causal Inference and Uplift Modeling a Review of the Literature,” by Pierre Gutierrez and Jean-Yves Gérardy and “Empirical Analysis of Model Selection for Heterogeneous Causal Effect Estimation,” by Divyat Mahajan et al.

Evaluation of causal models is an area of research that is still developing. As such, it still has many blindspots. For instance, the curves presented so far only tell you how good a model is in terms of ordering the CATE. I haven't found a good solution to checking if your model correctly predicts CATE. One thing that I like to do is to use the effect by quantile plot alongside the cumulative gain curve, since the first one gives me some

idea on how calibrated the model is while the second gives me an idea how well it orders the CATE. As for the normalized cumulative gain, it is just a zoom-in that makes visualization easier.

But I'll admit that this is not ideal. If you are looking for a summary metric like the R^2 or the MSE—both commonly used in predictive models—I'm sad to say I haven't found any good parallel to those in the causal modeling world. Here is what I did find, though—target transformation.

Target Transformation

It turns out that even though you can't observe the true treatment effect $\tau(x_i)$, you can create a target variable that approximates it in expectation:

$$Y_i^* = \frac{(Y_i - \hat{\mu}_y(X_i))(T_i - \hat{\mu}_t(X_i))}{(T_i - \hat{\mu}_t(X_i))^2} = \frac{Y_i - \hat{\mu}_y(X_i)}{T_i - \hat{\mu}_t(X_i)}$$

where μ_y is a model for the outcome and μ_t is a model for the treatment. This target is interesting because $E[Y_i^*] = \tau_i$. Notice how it looks a lot like the formula for the regression coefficient, with the numerator being the covariance between Y and T and the denominator, the variance of T . However, instead of using expectations to define those, it is computed on a unit level.

Since this target approximates the true treatment effect, you can use it to compute deviance metrics, like the mean squared error (MSE). If your model for CATE is good in terms of predicting the individual level effect τ_i , then the MSE of its prediction with respect to this target should be small.

There is a catch, though. This target will be incredibly noisy when close to the treatment average, where the denominator will tend toward zero. To fix that, you can apply weights that assign low importance to points where $T_i - \hat{\mu}_t(X_i)$ is small. For instance, you can weight the units by $(T_i - \hat{\mu}_t(X_i))^2$.

FORESHADOWING THE R-LOSS

There is a good theoretical reason for using those weights. You'll learn more about it when we talk about nonparametric Double/Debiased Machine Learning in [Chapter 7](#). For now, you'll have to take my word for it.

To code this target, you can simply divide the residuals of an outcome and a treatment model:

```
In [13]: X = ["C(month)", "C(weekday)", "is_hol:

y_res = smf.ols(f"sales ~ {'+'.join(X)
t_res = smf.ols(f"discounts ~ {'+'.join

tau_hat = y_res/t_res
```

Next, you can use it to compute the MSE of all your models. Notice how I'm also using weights like discussed previously:

```
In [14]: from sklearn.metrics import mean_squared_error

        for m in ["rand_m_pred", "ml_pred", "cate_pred"]:
            wmse = mean_squared_error(tau_hat,
                                      sample_weights,
                                      weights=sample_weights)
            print(f"MSE for {m}:", wmse)
```

```
Out[14]: MSE for rand_m_pred: 1115.803515760459
         MSE for ml_pred: 576256.7425385397
         MSE for cate_pred: 42.90447405550281
```

According to this weighted MSE, once again, the regression model used to estimate the CATE performs better than the other two. Also, there is something interesting here. The ML model performs worse than the random model. This is not surprising, as the ML model is trying to predict Y not τ_i .

SEE ALSO

Like I said before, the literature on evaluating causal models is still in its infancy. It's quite an exciting problem, where new methods are being proposed all the time. For example, in the paper "Intelligent Credit Limit Management in Consumer Loans Based on Causal Inference," the scientists from Ant Financial Services Group propose partitioning the units into groups that have similar covariates (they use over 6,000 groups!), pretending that outcome is the treatment effect plus some Gaussian random noise $\hat{y}_i = \hat{\tau}(x_i) + e_i$, computing outcome MSE in each group $N^{-1} \sum (y_i - \hat{y}_i)$ and averaging the results using the sample size in each group.

Predicting Y will only be good in terms of ordering or predicting τ_i when the effect is correlated with the outcome. This won't generally be the case, but there are some situations in which it might happen. Some of those are fairly common in business, so it is worth taking a look at them.

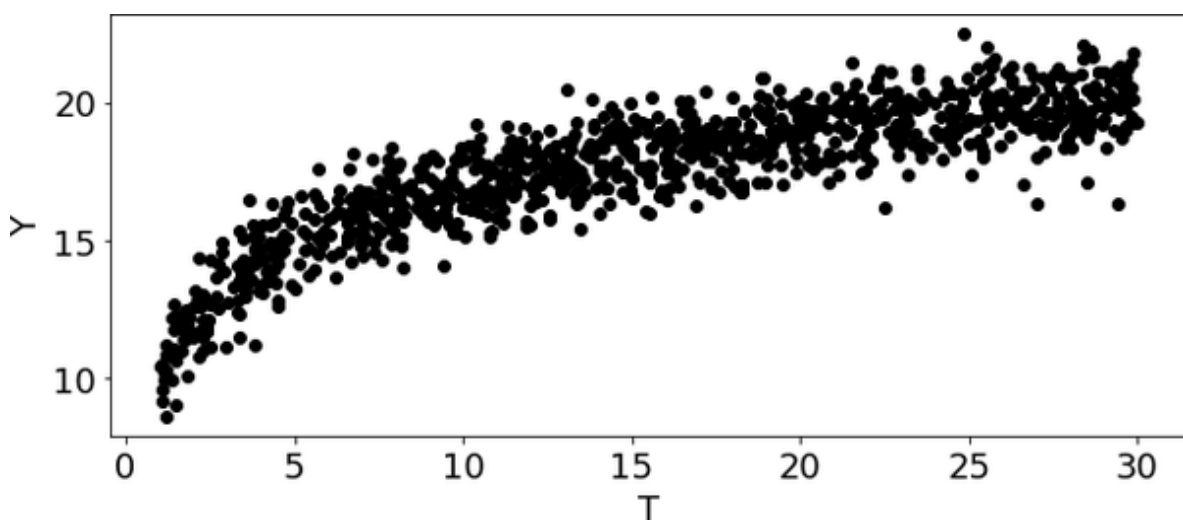
When Prediction Models Are Good for Effect Ordering

Like I said before, for a model that predicts Y to also be good at ordering the CATE, it must be the case that Y and the CATE $\tau(x_i)$ are also correlated. For instance, in the context of finding days where customers are more responsive to discounts in a restaurant, if the days when sales are high coincide with the days when people are more sensitive to discounts, then a model that predicts Y will also be good at ordering the effect of T on Y . More generally, this can happen when the treatment response function is nonlinear.

Marginal Decreasing Returns

When the treatment response function is concave, additional units of the treatment will have less and less effect. This is a very common phenomenon in business, as things tend to have a saturation point. For example, the number of sales can only go so high, even if you set a discount to 100%, as there are factors that limit the amount you can produce. Or, the effect of your marketing budget will eventually flatten out, since there are only so many customers you can advertise to.

A marginally decreasing treatment response function looks something like this:

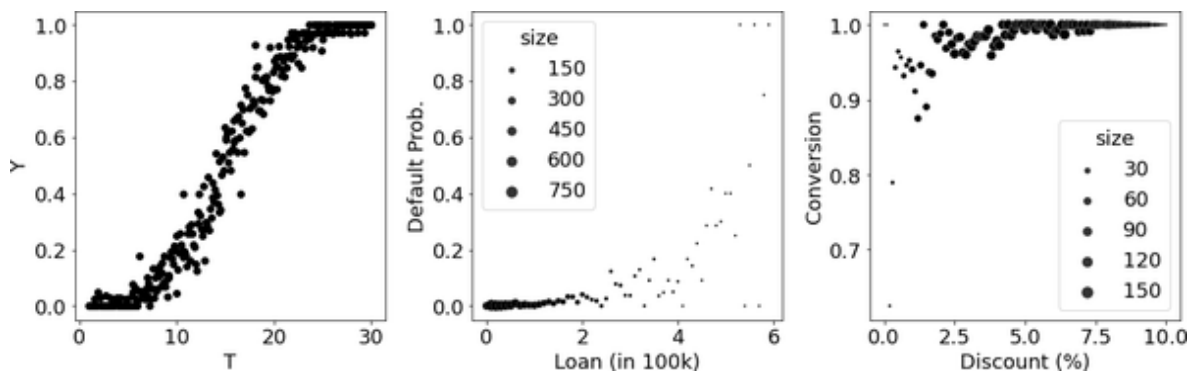


It is easy to see why, in this case, a model that is good at predicting the outcome Y can also be good at ordering the CATE: the higher the outcome, the lower the effect. Hence, if you take the model that predicts Y and sort your units by the

inverse of these predictions, you'll probably manage to get a decent CATE ordering.

Binary Outcomes

Another common situation where a model that predicts Y can be good for ordering the CATE is when the outcome is binary. In this case, $E[Y|T]$ has an S shape, flattening out at 0 and 1:



In most business applications, the data will be concentrated at one or the other end of this S-shaped function. For example, in banking, only a small fraction of customers will default on their loans, which means you would be mostly to the left side of this curve, where it looks exponential. As a result, if you have a model that predicts customer default, there is a good chance that customers with higher predictions will also be more sensitive to the treatment. Intuitively speaking, those are customers close to the tipping point between not defaulting and defaulting. For them, a small change in the treatment can make all the difference.

In contrast, let's say you work at an online shopping business where most of the customers that enter your site do buy something (convert). In this case, you are more to the right of the S-shaped curve. Hence, if you have a model that predicts conversion, there is a good chance that the same model can also order the effect of something like discounts. The higher the chances of conversion, the lower the effect size. That's because at the right side, the S curve looks a bit like the marginally diminishing returns case you saw before.

In general, when the outcome is binary, the closer you are to the middle—that is, to $E[Y|X] = 50\%$ —the higher the effect will be.

PRIORITIZING VACCINES

You saw how binary outcome induces a nonlinearity in the treatment response function, which allows you to use the predictive value of the outcome to allocate the treatment. A very interesting application of this principle was seen in the COVID-19 pandemic. In 2021, the world managed to deliver its first batch of approved COVID-19 vaccines to the general public. Back then, a crucial question was who should receive the vaccine first. This is, not surprisingly, a heterogeneous treatment effect problem. Policymakers would like to vaccinate those who would benefit the most first. In this situation, the treatment effect is avoiding death or hospitalization. So, whose death or hospitalization decreased the most when given a shot? In most countries, they were the elderly and those with prior health conditions (comorbidities). Now, these are the people that are more likely to die when getting COVID-19. Also COVID-19 mortality rate is thankfully much lower than 50%, which puts you to the left of logistic function. In this region, by the same context we made for default rates, it would make sense to treat those with a high baseline probability of death when getting COVID-19, which are precisely the groups mentioned earlier. Is this a coincidence? Maybe. Keep in mind that I'm not a health expert, so I might be very wrong here. But the logic makes a lot of sense to me.

When the treatment response function is nonlinear, like in the binary outcome or in the case where the outcome is marginally decreasing, a predictive model might yield a good ordering of the CATE. Still, this doesn't mean that it will be the best model nor that it can't be outperformed by a model that aims at directly predicting the CATE. Moreover, even though such a model might order the treatment effect, it does not predict that treatment effect. This is only OK if all you care about is sorting units by their sensitivity to the treatment. But in case your decision making depends on correctly estimating the CATE, additional effect by group estimation will be required.

SEE ALSO

Sometimes outcome prediction can outperform CATE prediction because CATE tends to be very noisy. Fernández-Loría and Provost discuss this further in their paper, "Causal Classification: Treatment Effect Estimation vs. Outcome Prediction."

Speaking of that, I think it is worth spelling out how you could use the CATE for decision making. You probably already have a good idea how to do that, but maybe I have some piece of advice you haven't thought about.

CATE for Decision Making

When the treatment is binary, the decision-making process is pretty straightforward. You essentially care about who responds positively to the treatment. If you have an unlimited

supply of treatments, then all you have to do is treat everyone whose CATE is positive. If you don't have a model that predicts the CATE, but you do have one that orders it—as in the case of the predictive models discussed in the previous section—you can use the effect by model quantile plot. Just partition your data by quantiles of your model, estimate the treatment effect in each quantile, and treat everyone up to the point where the effect is still positive.

If you don't have an unlimited supply of the treatment, then you need to add a second rule. Not only will you treat only those with positive effect, but also those with the highest CATE. For example, if you only have 1,000 units of the treatment, you probably want to treat the top 1,000 units, according to some CATE ordering model, provided that they all have positive effect.

If the treatment is continuous or ordered, things get a bit more complicated. You now have to decide not only who to treat, but also how much to treat. This is very business specific. Each problem will have its own treatment response function to optimize. This means I can't give you very detailed guidelines on how to do it, but I can walk you through a typical example.

Consider once again the problem of deciding how much discount to give each day on a chain of restaurants. Since deciding how much discount to give is just another way of deciding what price to charge (

$Price = Price_{base} * (1 - Discount)$), let's reframe that

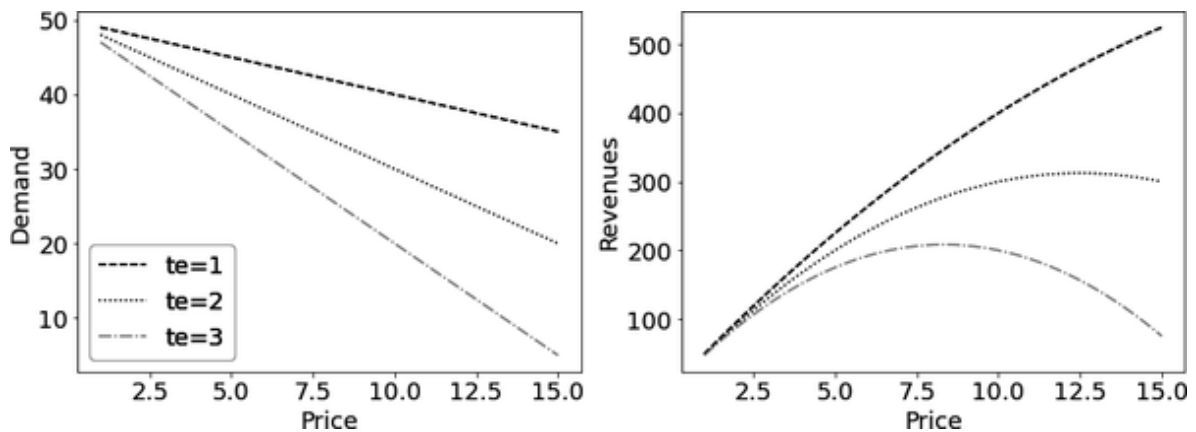
problem as a price optimization one. In all business problems there is a cost (even if not monetary) and a revenues function. Let's say that the revenue of the restaurant is given by the following equations:

$$Demand_i = 50 - \tau(X_i)Price_i$$

$$Revenue_i = Demand_i Price_i$$

Revenue on a day i is just the price times the number of meals (demand) the restaurant serves. However, the number of meals people are willing to buy on a particular day is inversely proportional to the price charged on that day. That is, it has a component $-\tau(X_i)Price_i$, where $\tau(X_i)$ is how sensitive customers are to price increases on that day (notice that this depends on the date-specific features, X_i). In other words, that sensitivity is the conditional average treatment effect of price on demand.

If you plot the demand curve for different values of τ , you will see that τ is nothing more than the slope of the demand curve. If you take the demand curve and multiply it by the revenues curve, you would get a quadratic shape. In this curve, the day in which customers are less sensitive to price ($\tau = 1$) peak at a latter price value:



Next, suppose that you spend 3 dollars in order to produce your meal. This means that the cost is simply the quantity you produce, q , times 3:

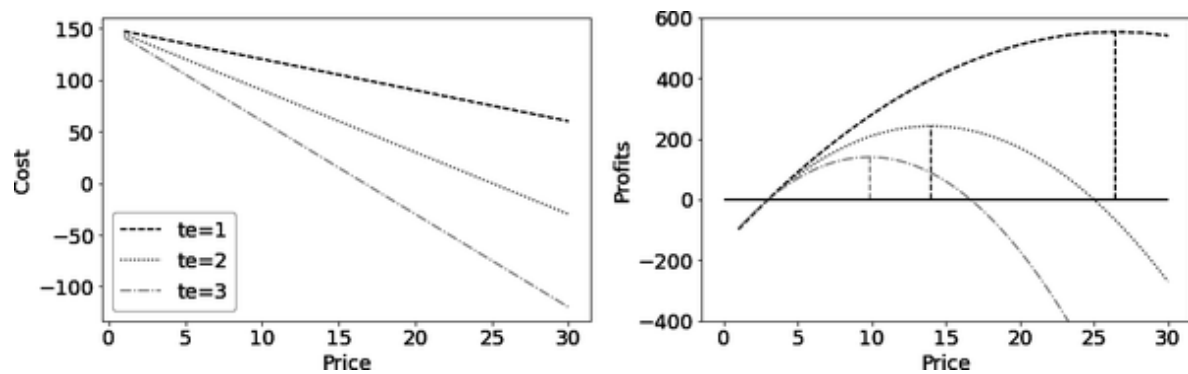
$$Costs(q_i) = 3q_i$$

Keep in mind that the cost equation doesn't depend on the treatment effect directly, but if you recall that the quantity produced is just the quantity of customer orders—that is, the demand—then, the higher the price, the lower the cost, as customers will demand fewer meals.

Finally, once you have both revenues and costs, you can combine them to get the amount of profit as a function price:

$$Profit_i = Demand_i * Price_i - Cost(Demand_i)$$

If you plot the profits by price for different values of τ_i , you'll see that each yields a different optimal price. The lower τ_i , the less sensitive customers are to price increase, which allows the restaurant to increase prices in order to make more profits:



Economists will quickly realize that this is the famous problem of the firm. Setting the marginal cost equal to the marginal revenue and isolating price gives a numerical solution to the profit maximizing price:

$$(P(50 - \tau(X)P))' = (3(50 - \tau(X)P))'$$

$$P^* = \frac{3\tau(X) + 50}{2\tau(X)}$$

Notice that, in this case, the only unknown is the effect of price on demand, $\tau(X)$. So, if you can estimate it using a model that predicts CATE, you can convert that CATE prediction to the optimal price.

Again, this is highly dependent on the form of the revenue and cost curve, which in turn depends a lot on your business. But, in general, almost any treatment you care to optimize has an upside—revenues, in this example—and a downside—costs, in this example. To use CATE for deciding the level of a continuous treatment you have to understand how it impacts both of those sides.

CORNER SOLUTION

In some rare situations, the treatment level that optimizes a business is none at all or the maximum allowed level. For instance, let's say that your local government sets a price cap on the product you are selling. That cap is below the price that would maximize your profits. In this case, the optimal price would simply be the maximum allowed by the government. This situation is rare, however. In most cases, corner solutions arise when hidden prices are not taken into account. For example, if you are trying to optimize cross-sell emails, you can argue that the cost of sending an email is negligible, so you should just go ahead and send it to everyone. But I would argue back saying that you are not taking into account the costs in terms of customer attention: if you spam your customers, eventually they will get tired of you and unsubscribe to your emails, which will cost you future sales that would come through the email channel. These hidden costs are much harder to take into account, but it doesn't mean that they aren't there. In fact, finding good proxies for those costs tend to be an invaluable data science task.

Key Ideas

This chapter introduces the idea of treatment heterogeneity. The key insight is that each unit i can have a different

treatment effect τ_i . If you knew this effect, you could use it to better allocate the treatment among units. Sadly, due to the fundamental problem of causal inference, this effect is unobservable. Still, if you assume that it depends on observable features of the units, $\tau(x_i)$, then you can make some progress; namely, you can go from estimating the average treatment effect to estimating the conditional average treatment effect (CATE):

$$CATE = \frac{\partial}{\partial t} E[Y(t)|X]$$

So, even though the treatment effect is not observed at a unit level, you can still estimate group effects. A simple way of doing that is with linear regression, by including an interaction term between the treatment and the covariates:

$$y_i = \beta_0 + \tau_0 T_i + \tau X_i T_i + \beta X_i + e_i$$

Estimating this model would give you the following CATE estimate:

$$\widehat{CATE} = \tau_0 + \tau X_i$$

Next, you saw some ideas on how to pair cross-validation with CATE evaluation techniques in order to evaluate your CATE estimates. Since the CATE is not defined for a single unit, you had to rely on group-specific metrics, like the effect by quantile curve or the cumulative gain curve. If that is not enough, you could also define a target that approximates the

individual-level treatment effect and use that to calculate deviance metrics, like the MSE.

Finally, it's worth emphasizing that everything discussed in this chapter hinges on the fact that the CATE, a causal quantity, can be identified from the conditional expectation, a statistical quantity recoverable from data:

$$\frac{\partial}{\partial t} E[Y(t)|X] = \frac{\partial}{\partial t} E[Y|X, T = t].$$

Without that, the idea of CATE as a group effect you can estimate no longer holds, which is why randomized data is so important for CATE estimation problems, even if just for evaluating your treatment heterogeneity models.